2026 전남광주 기업 AX 혁신 챌린지 · 해커톤 사전 교육

AIStream

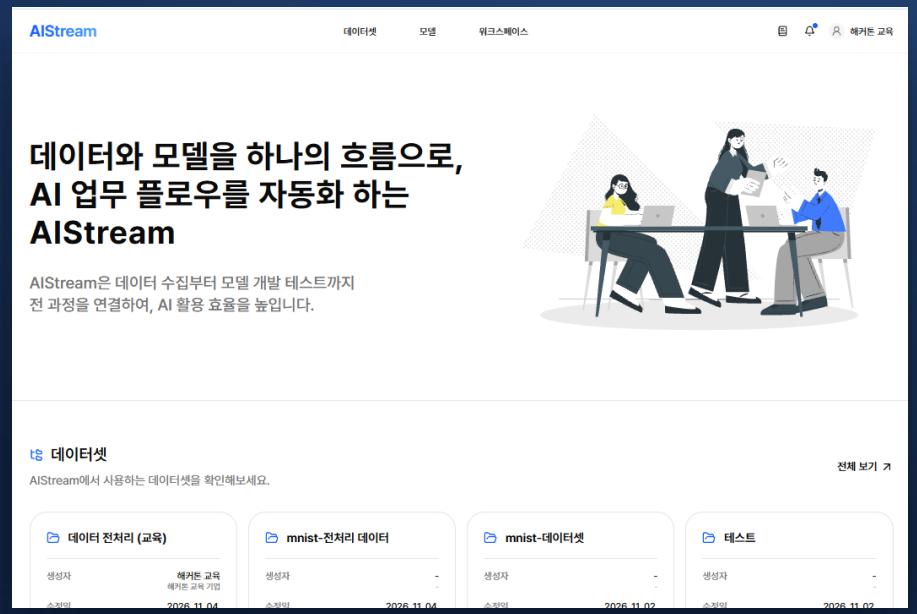
데이터에서 모델 학습 · 배포 · 추론까지

MLOps 개념 · 안전모 활용사례 · MNIST 실습 · 서빙 API 활용

AIStream 1 · AIStream 2

접속 주소 aiopen.jntp-ax.or.kr

코드 없이 화면으로 진행



AIStream 1 · 2, 이렇게 진행해요

01 설명

- MLOps가 무엇인가
- 핵심 용어 6가지
- 서빙과 API
- AIStream 구조

02 예시

- 안전모 활용사례
- 사용 모델 · 파이프라인
- 워크플로우 · 노드 설정 · 실행
- 서빙 URL로 추론

03 실습 1

- 로그인 · 워크스페이스 입장
- MNIST 워크플로우 생성
- 노드 설정 · 파라미터 확인
- 실행

04 실습 2

- URL · 연결 구조
- 내 서빙 URL 확인
- /docs 읽기 · 그려서 숫자 추론
- Codex로 내 서비스 만들기

05 해커톤 사전 안내

- 트랙 고르기 (A · B)
- A트랙 제공 서빙 3종
- 배포 목록에서 서빙 찾기
- 부록 · Q&A

AIStream 1 01 · 02 → 휴식 → AIStream 2 03 · 04 → 05 해커톤 사전 안내 · Q&A

오늘의 목표 **MNIST 모델을 직접 학습 · 배포하고, 서빙 URL로 숫자를 맞혀 보기**

모델은 '서비스'가 되어야 쓸모가 생깁니다

💻 내 노트북 속 모델

- 정확도 92%, 그런데 다른 사람은 어떻게 쓰지?
- 공유 방법 없음 · 재학습은 처음부터
- 설정 기록 없음 · GPU 확보 어려움

만들었지만 쓰이지 못하는 모델

🌐 누구나 호출하는 AI 서비스

- URL 하나로 웹 · 앱 · 내 서비스에서 호출
- 버튼으로 재학습
- 버전 · 결과 자동 기록 · GPU는 필요할 때만

서비스로 계속 쓰이는 모델

AI 모델은 만드는 것보다 **계속 쓰이게 하는 것**이 더 어렵습니다. 그 방법이 MLOps예요.

MLOps = Machine Learning + Operations

AI 모델을 개발 · 배포 · 운영 · 개선하는 전 과정을 **반복 가능하고 자동화된 흐름**으로 만드는 방법입니다.

1

01

데이터 준비

수집 · 정제 · 버전 관리

2

02

학습 · 실험

파라미터와 성능 기록

3

03

배포 (서빙)

모델을 API 서비스로

4

04

모니터링

호출량 · 자원 확인

성능이 떨어지거나 새 데이터가 쌓이면 다시 데이터 준비부터 — **재학습 루프**

재현성
같은 결과를 언제든 다시

자동화
사람 손을 거치는 단계 줄이기

협업
데이터 · 모델 · 결과를 팀이 공유

운영
배포 후에도 상태를 계속 확인

기본 개념 5가지 — 빵집으로 이해하기

데이터가 모델이 되고, 모델이 서비스가 되기까지 **네 단계** — 그리고 이 흐름을 계속 굴리는 방식이 **MLOps**

1 전처리

빵집: 재료 손질

원본 데이터를 모델이 먹을 수 있게 정리

- 깨진 사진 · 빠진 정답 걸러내기
- 크기 · 형식 맞추기
- 학습 / 검증 / 테스트로 나누기

오늘 MNIST에서
70,000장 → 학습 54,001 · 검증 5,999 · 테스트 10,000

2 학습

빵집: 레시피 연구

정답 붙은 데이터를 반복해서 보며 규칙을 스스로 찾기

- 예측 → 정답과 비교 → 조금씩 수정
- 학습 조건 = 파라미터 (epochs · batch)
- 결과는 MLflow에 자동 기록

오늘 MNIST에서
CNN · 10번 반복 → 검증 정확도 98.6%

3 서빙

빵집: 매장 문 열기

학습된 모델을 24시간 대기하는 API로 띄우기

- 서빙 URL = 매장 주소
- /docs = 메뉴판 · /health = 영업 중?
- 배포 노드 하나로 자동 생성

오늘 MNIST에서
.../jntp-cpu/SV-88718065

4 추론

빵집: 주문 받아 내주기

새 사진을 보내 답을 받기 (0.1초 안팎)

- 학습 = 공부 · 추론 = 시험
- 결과 + 확신도(0~1)를 돌려줌

오늘 MNIST에서
숫자 그림 → "7" · 확신도 0.99

MLOps

① ~ ④를 **버튼 한 번으로 다시 돌릴 수 있게** 만드는 운영 방식
현장 데이터가 바뀌면 → 새 데이터로 **재학습 → 재배포** (↻ 다시 ①부터)

MLOps가 있을 때와 없을 때

구분	MLOps 없이	MLOps (AIStream)와 함께
재현성	“그때 그 모델 어떻게 만들었지?”	**데이터 · 파라미터 버전이 남아 언제든 재현**
협업	파일을 메신저로 주고받음	**워크스페이스에서 데이터 · 모델 · 결과 공유**
배포	개발자가 서버를 직접 세팅	**배포 노드 하나로 API 자동 생성**
자원	GPU 서버를 통째로 점유	**작업 단위로 CPU · GPU 할당 후 회수**
운영	문제가 생겨도 원인 파악이 어려움	**실행 이력 · 모니터링으로 추적**

MLOps는 누가 쓸까? — 모델 하나에 여러 역할이 함께

빵집에 비유하면 재료 → 레시피 → 주방 → 매장 → 손님까지, 각자 맡은 단계를 **한 흐름**으로 이어 줍니다

누가	하는 일	빵집 비유	AIStream에서
데이터 엔지니어	데이터를 모으고 정리해 버전으로 관리	재료 담당	**데이터셋 · 전처리 노드**
데이터 사이언티스트	모델을 고르고 파라미터를 바꿔 가며 실험	레시피 연구원	**학습 노드 · MLflow**
ML 엔지니어	잘 된 모델을 서빙(API)으로 배포 · 재학습 자동화	주방장	**워크플로우 · 배포 관리**
플랫폼 엔지니어	GPU 등 자원 배분 · 플랫폼 운영 · 장애 대응	건물 · 설비 관리	**자원 한도 · 모니터링**
서비스 개발자	서빙 URL을 불러 웹 · 앱 화면 만들기	홀 직원	**서빙 URL · /docs**
현업 담당자	결과를 업무에 쓰고 "잘 안 맞는다" 알려 주기	손님	**서비스 화면**
관리자 · 의사결정자	비용 · 성과 확인, 투자 판단	사장님	**크레딧 · 사용 현황**

꼭 알아야 할 MLOps 용어 6가지

예시와 실습의 AIStream 화면에서 **이 용어들이 그대로** 다시 나옵니다.

01

데이터셋 버전

학습에 쓰는 데이터. 바뀔 때마다 v1, v2 같은 버전으로 남깁니다.

02

워크플로우 파이프라인

전처리 · 학습 · 배포 작업을 노드로 연결한 실행 흐름입니다.

03

실험 추적 MLflow

학습 때 쓴 파라미터와 성능 지표를 자동으로 기록합니다.

04

모델 레지스트리

학습된 모델을 버전별로 보관하는 저장소입니다.

05

서빙 Serving

모델을 항상 대기하는 API 서버로 띄우는 것입니다.

실습 2에서 직접 호출

06

서빙 URL 엔드포인트

서빙된 모델을 호출하는 주소. 해커톤의 핵심 재료입니다.

서빙과 API — 사진을 보내면 결과가 돌아옵니다

내 쪽

💻

내 서비스

웹 · 앱 · 내 서비스 화면
사진을 올리고 결과를 화면에 표시

① 요청
이미지 파일을 보냄

② 응답
판별 결과를 돌려받음

AIStream 배포 관리에서 운영

🚀

서빙된 모델

안전모 판별 추론 (YOLO)
MNIST 숫자 판별 (CNN)

무엇	무엇을 찾았는지 (안전모 / 숫자)
확신도	얼마나 확신하는지 (0~1)
위치	사진 속 어디에 있는지 (안전모)

AIStream = MLOps 전 과정을 담은 플랫폼

AIStream 메뉴	하는 일	MLOps 단계
데이터셋	데이터 탐색 · 업로드 · 버전 관리	데이터 준비
모델	모델 템플릿 · 파라미터 · 활용 프리셋 조회	모델 선택
워크스페이스 › 워크플로우 관리	노드를 연결해 전처리 · 학습 · 배포 실행	파이프라인
워크스페이스 › 산출물 관리	결과 데이터셋과 등록 모델 버전 보관	레지스트리
워크스페이스 › 배포 관리	서빙 URL · Swagger · 모니터링	서빙 · 운영

함께 동작하는 연계 시스템 MLflow 실험 기록 · Airflow 실행 스케줄 · MinIO 파일 저장 · Grafana 자원 현황

AIStream 1 · 02 예시 1

안전모 활용사례로 MLOps 흐름 살펴보기

현장 데이터 → 워크플로우 → 노드 설정 → 실행 → 배포 관리 → 서빙 URL로 추론

STEP 6개 | 강사 시연 | YOLO11n 객체 탐지



활용사례: 현장 사진으로 안전모 미착용 찾기

조선소 작업장 사진으로 **YOLO11n 객체 탐지 모델**을 AIStream에서 학습 · 배포했습니다.

문제	넓은 작업장에서 모든 작업자의 안전모 착용 여부를 관리자가 매 순간 확인하기 어려움
데이터	작업장 안전모점검장치 카메라 사진 **1,901건** (jpg 960×272 + YOLO 라벨 txt · 약 100MB)
정답	**hat** 안전모 착용 / **no_hat** 안전모 미착용 — 사진 속 위치를 박스로 표시
모델	YOLO11n — 가볍고 빠른 객체 탐지 모델 (위치를 찾고 착용 여부를 구분)
학습 설정	epochs **50** · 이미지 크기 **640** · batch **16** · 결과는 MLflow에 자동 기록
결과	전처리 1분 17초 + 학습 11분 38초 + 배포 5초 = **약 15분** → 추론 API → '안전모 착용 감지' 웹 서비스



원본 — 작업장 안전모점검장치 카메라



AI 검출 결과 — 안전모 미착용 0.69

YOLO 모델이란? (You Only Look Once)

사진을 **한 번만 보고** 여러 물체의 **위치(박스)와 종류**를 동시에 찾는 **객체 탐지** 모델
빨라서 CCTV 같은 실시간 영상에 적합 · YOLO11n = 11번째 버전의 **n(nano, 가장 가벼운 크기)**

MLOps 포인트 새 현장 데이터가 쌓이면 같은 워크플로우를 다시 실행하는 것만으로 모델 갱신

오늘의 시나리오: 작업자 안전모 착용 판별 AI

워크플로우 '해커톤 교육용 안전모 워크플로우'의 실제 노드 구성입니다

1 전처리

preprocess/helmet-image

IN 안전모 이미지 데이터셋
OUT 전처리된 데이터셋

- 사진 ZIP + 정답 ZIP 짝 확인
- 깨진 사진·빈 정답 검사

이미지 1,901장 · 라벨 2,051개

2 학습

ml/yolo-v11n

IN 전처리 데이터 + YOLOv11n 베이스 모델
OUT 학습된 모델

- 기존 YOLO 가중치에서 추가 학습
- 예측과 정답 비교로 반복 조정

epochs 50 · batch 16 · imgsz 640

3 배포

serving/yolo-v11n

IN 학습된 모델
OUT 서빙 '안전모 판별 추론'

- 대상 위치·착용 여부 예측

GPU b200-1g-23gb · 16C32GB

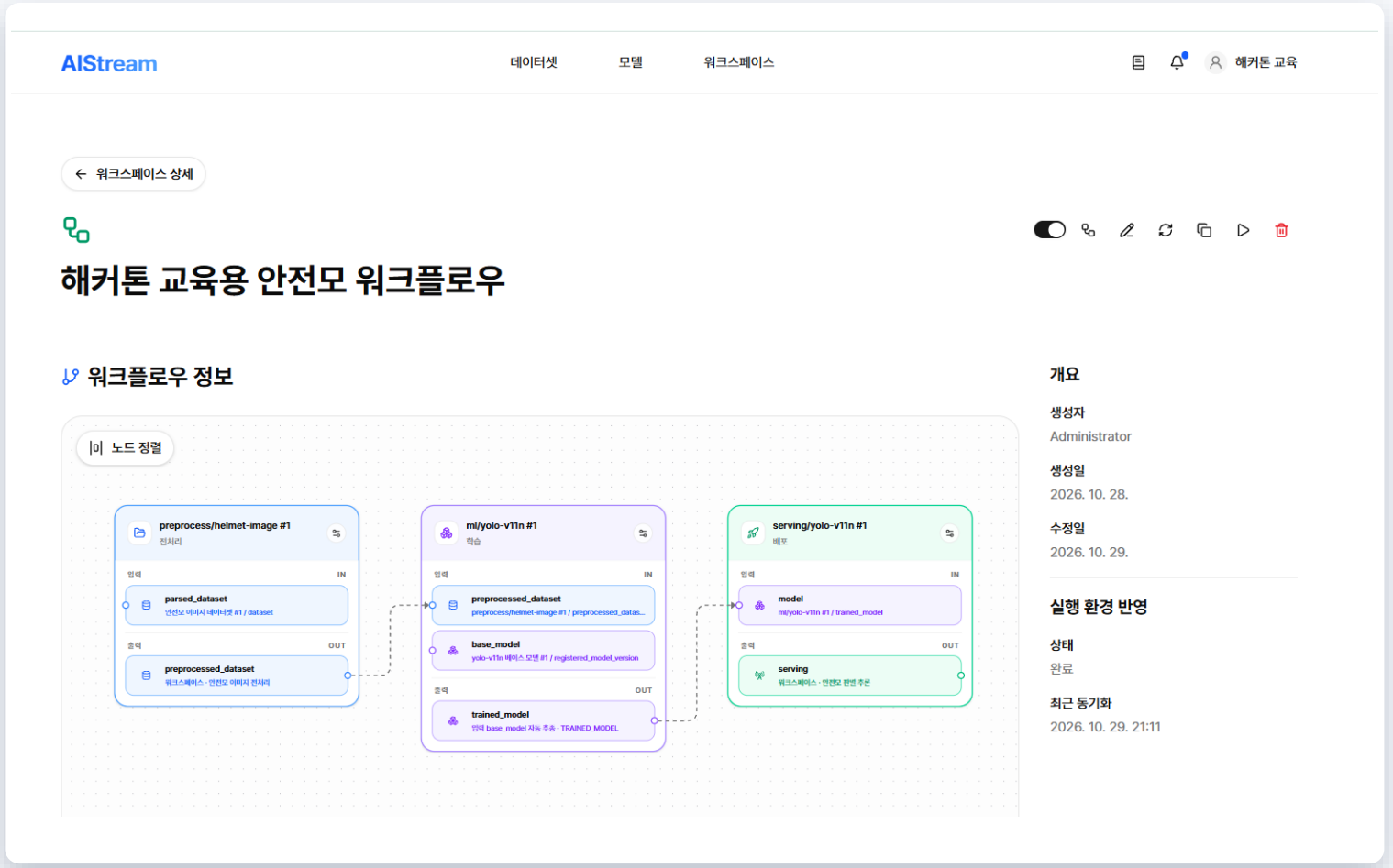
최적 모델 best.pt → MLflow

STEP 01

워크플로우는 노드의 연결입니다

1. **파란 노드 · 전처리** — preprocess/helmet-image
2. **보라 노드 · 학습** — ml/yolo-v11n
3. **초록 노드 · 배포** — serving/yolo-v11n
4. 상단 아이콘: 활성 스위치 · 편집 · 복제 · ▶ 실행

✓ **확인** 앞 노드의 출력(OUT)이 뒤 노드의 입력(IN)으로 **자동 전달** — 파일을 옮길 필요가 없어요



STEP 02

노드 설정: 입력 · 출력 · 파라미터 · 자원

1. 노드를 **더블클릭**하면 설정 창이 열림
2. 필수 입력과 출력 대상을 지정
3. 파라미터와 자원 프로필 확인 → **[설정 확인 완료]**

! 실행 전 확인: 필수 입력이 모두 연결됐는지 · 출력 대상을 지정했는지 · 분당 실행 비용과 크레딧 잔액

! GPU가 필요 없는 노드는 **[No GPU]** 선택

설정 창의 4가지 항목

입력
데이터셋 버전, 베이스 모델

출력
새 데이터셋, 등록 모델

파라미터
epochs, batch, imgsz 등

자원
CPU/메모리 프로필, GPU 프로필

안전모 학습 노드(ml/yolo-v11n)의 실제 파라미터

epochs 50	전체 데이터를 50번 반복해서 학습
batch 16	한 번에 16장씩 묶어서 학습
imgsz 640	이미지 크기를 640px로 맞춤

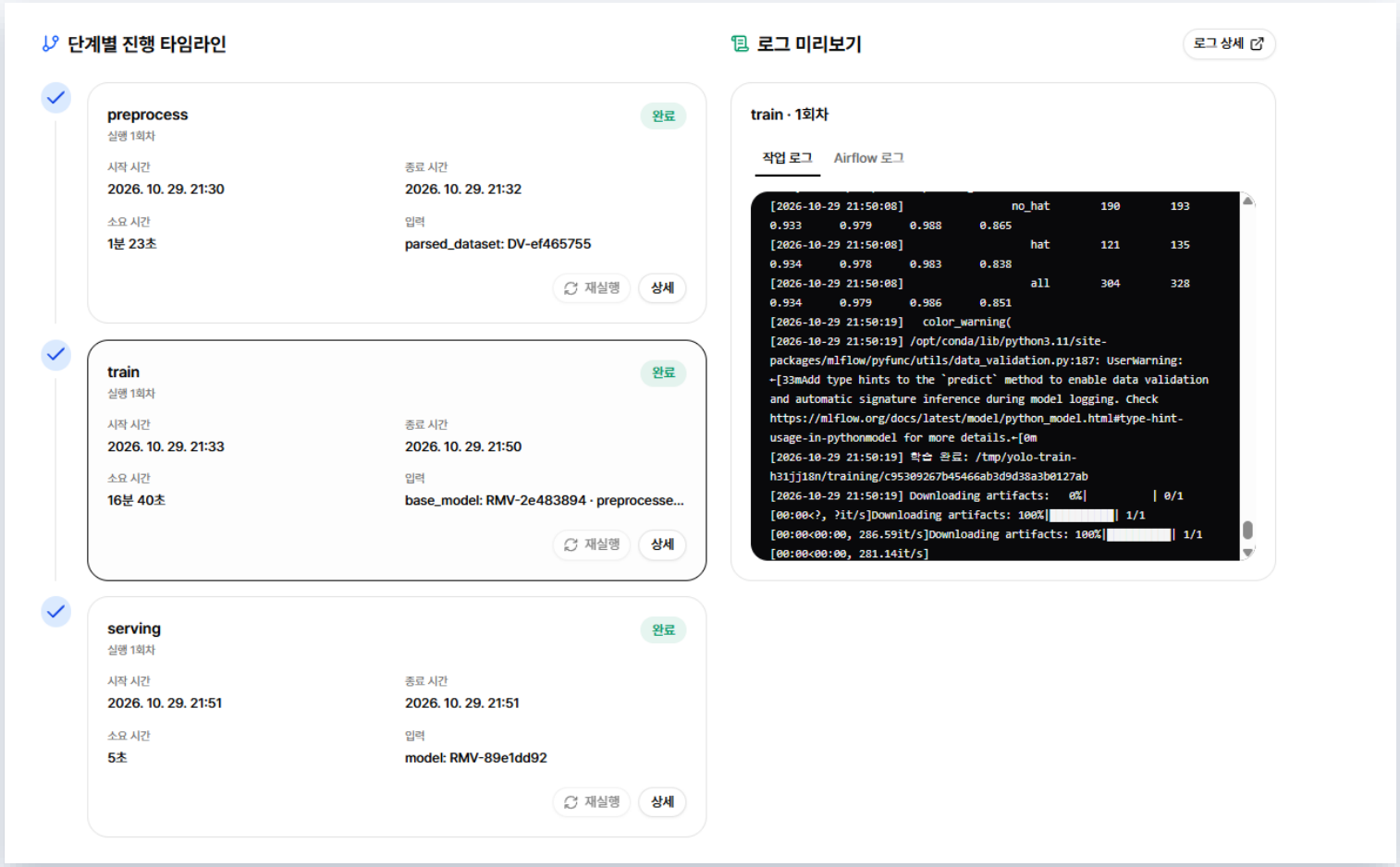
STEP 03

▶ 실행하고 단계별 진행 확인하기

1. 워크플로우 상단 ▶ **[워크플로우 실행]** 클릭
2. 아래 **실행 이력**에서 회차(#3) 클릭
3. 왼쪽 **단계별 진행 타임라인**에서 단계마다 **완료** 여부 확인
4. 오른쪽 **로그 미리보기**에서 **[Airflow 로그]** 탭을 누르면 실시간 작업 로그 확인

✓ **확인** 실행 중에도 **Airflow 로그**로 지금 어떤 작업이 진행되는지 실시간으로 볼 수 있어요

✓ **확인** #3 실행 결과: 사용 크레딧 220.95 · 상태 **완료**



STEP 04

배포 관리에서 서빙 URL 찾기

1. 워크스페이스 **[배포 관리]** → 배포 목록
2. **'안전모 판별 추론'** 클릭
3. 운영 상태 '실행 중', 승인 상태 '승인 완료' 확인
4. **서빙 URL(화면 표기: 서비스 URL) 옆 복사 아이콘 클릭**

✓ **확인** 배포(serving) 노드가 성공하면 배포 목록에 서비스가 생깁니다

! **60분간 호출이 없으면** 자동 일시 정지 → 첫 호출이 조금 느릴 수 있어요

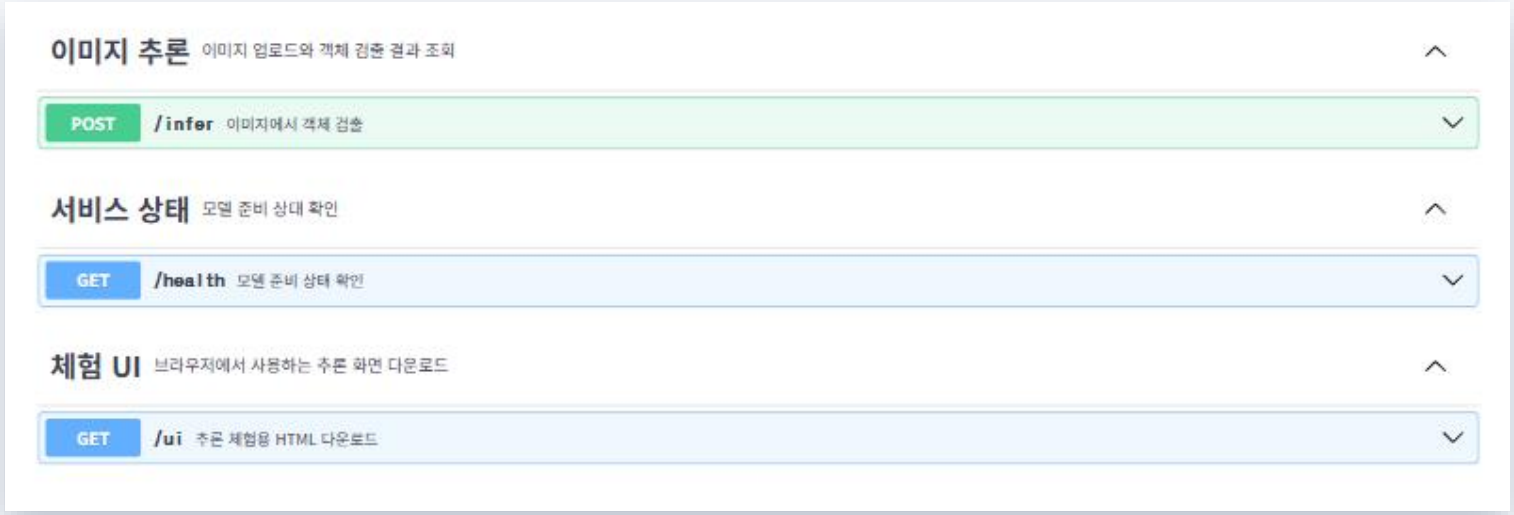


STEP 05

서빙 URL의 /docs 읽어 보기

1. 배포 상세에서 **[Swagger 열기]** 클릭
2. **POST /infer · GET /health 기능 목록 확인**
3. /infer를 펼쳐 보내는 형식 · 받는 형식 읽기
4. 실행 없이 읽기만 — 추론은 다음 단계 화면에서

/health	모델 준비 상태 확인 · 정상이면 ok
/docs	Swagger 문서 · 기능 설명서
/infer	이미지를 보내 추론

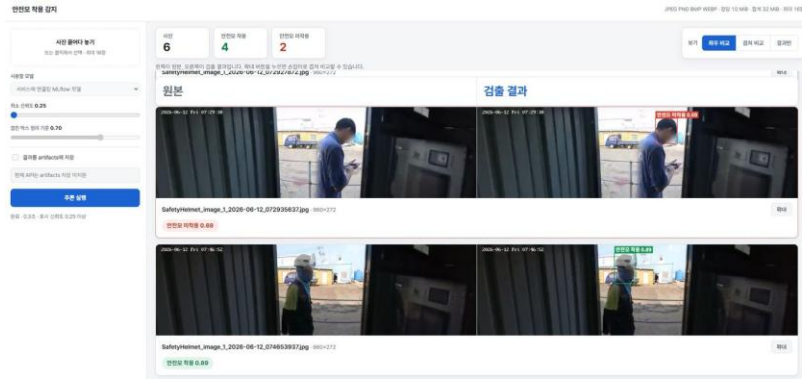


STEP 06

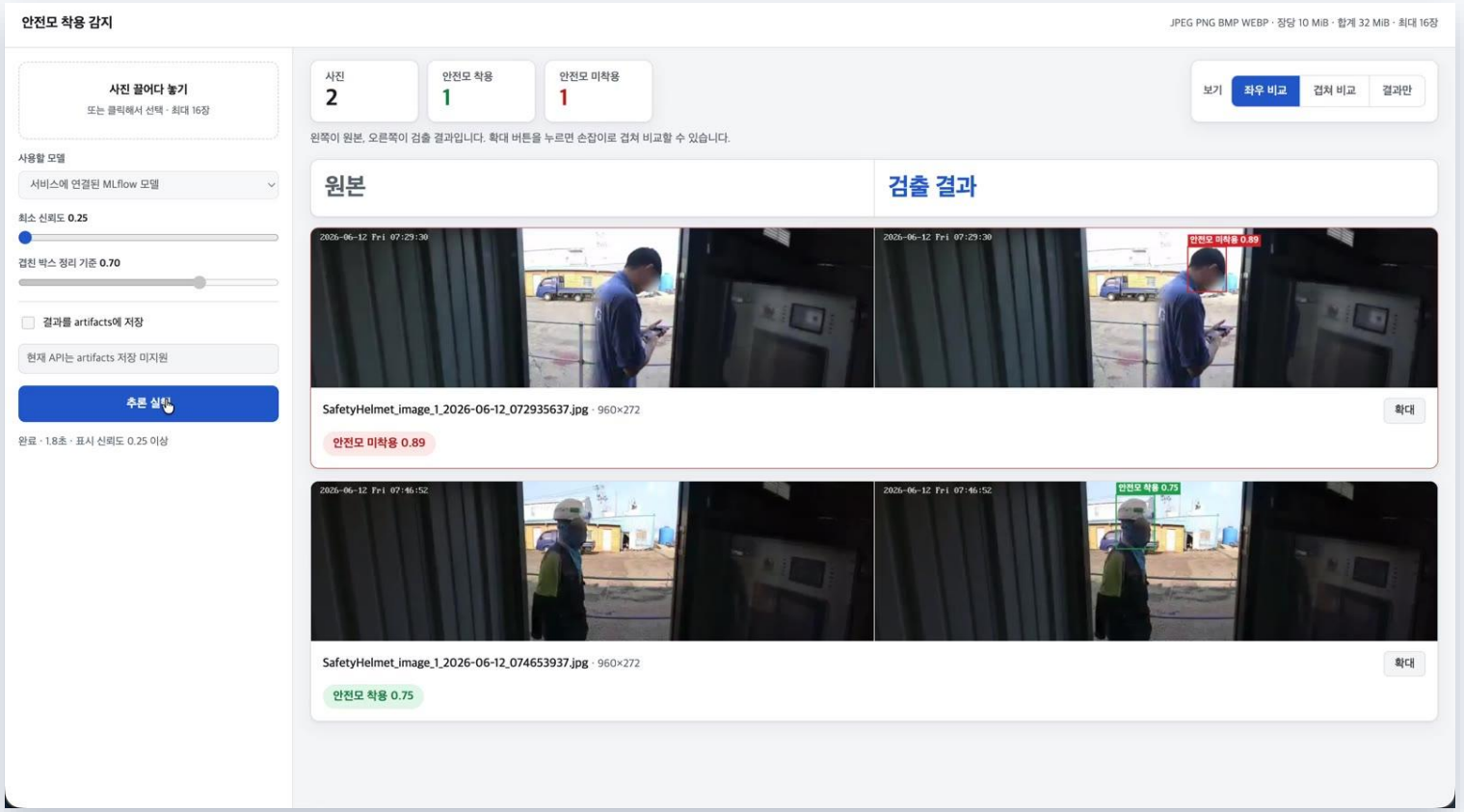
서빙 URL로 직접 추론해 보기

1. 배포 상세에서 **서빙 URL** 복사
2. '안전모 착용 감지' 화면에 **서빙 URL 입력** → [확인]
3. 현재 사진 선택 (최대 16장) → **[추론 실행]**
4. 원본과 검출 결과를 나란히 비교

✓ **확인** 사진마다 **착용 / 미착용 + 신뢰도**, 위에는 착용 · 미착용 건수



결과 화면 — 미착용 0.69 · 착용 0.89



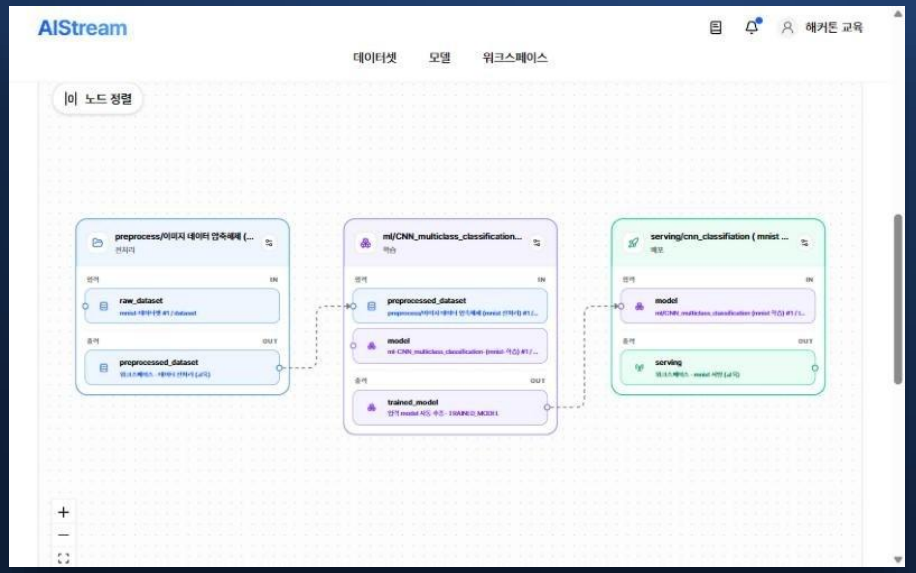
▶ 시연 영상 (23초) · 서빙 URL 입력 → 사진 선택 → 추론 실행 → 결과 확인

AIStream 2 · 03 실습 1

MNIST로
직접 따라해보기

워크스페이스 입장 → 워크플로우 생성 → 프리셋 · 노드 설정 → 파라미터 → ▶ 실행

STEP 9개 | CNN 모델 | 손글씨 숫자 0~9 분류



실습 시작 전: AIStream 접속하고 로그인하기

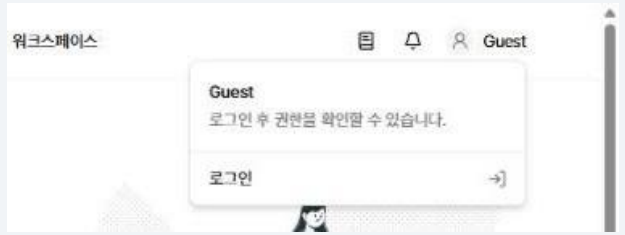
크롬 주소창에 접속 주소를 입력하고, **배부받은 계정**으로 로그인합니다

1 접속하기



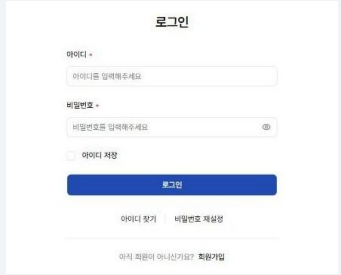
우측 상단이 Guest면 로그인 전

2 Guest → [로그인]



Guest 클릭 → 메뉴의 [로그인]

3 아이디 · 비밀번호 입력



포털 로그인 화면에서 배부된 계정 입력

접속 주소 **https://aiopen.jntp-ax.or.kr**

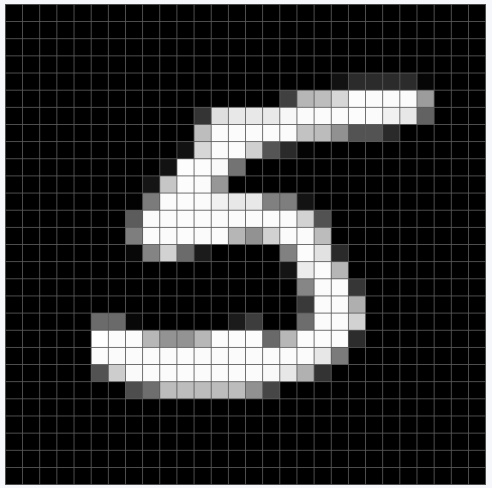
로그인 완료 → 우측 상단에 내 이름 · 해커톤 교육 워크스페이스 1개

MNIST란? AI 공부의 첫 연습 문제

손글씨 숫자 사진 7만 장을 모아 둔 공개 데이터셋 — 전 세계 AI 입문자가 가장 먼저 다루는 **'AI의 Hello World'**



실제 MNIST 원본 이미지 — 숫자별 14장 (같은 숫자도 쓰는 사람마다 모양이 달라요)



한 장을 확대하면 **28 × 28 = 784칸**
칸마다 밝기 **0(검정) ~ 255(흰색)**

정답(라벨) **5**

한눈에 보는 MNIST

무엇	0~9 **손글씨 숫자** 흑백 사진 + 사진마다 **정답(라벨)**
크기	**28 × 28 픽셀** — 한 장이 밝기 숫자 784개
수량	학습용 **60,000** + 시험용 **10,000**장
출처	미국 표준기술연구소(NIST) 손글씨를 다듬어 1998년 공개 — **M**odified **NIST**
왜 쓰나	가볍고 정답이 분명해 **CPU로 몇 분**이면 학습 → 전 과정 연습에 딱

오늘 실습: 이 사진들로 **'사진 한 장 → 0~9 중 무엇?'**을 맞히는 분류 모델을 학습하고 서빙 URL로 불러 봅니다.

실습 시나리오: 손글씨 숫자(MNIST) 분류 AI

워크플로우 'mnist 교육'(해커톤 교육 워크스페이스)의 실제 노드 구성입니다

1 전처리

이미지 데이터 압축해제 (mnist 전처리)

IN mnist-데이터셋 (PNG 70,000장)
OUT 학습 · 검증 · 테스트로 나눈 데이터셋

- 압축파일 mnist_png.tgz 1개
- 폴더명 = 정답 (0~9)
- 28×28 흑백 · 픽셀 0~1 변환

학습 54,001 · 검증 5,999 · 테스트 10,000장

학습 · 검증 · 테스트로 분리

2 학습

CNN 이미지 분류 (mnist 학습)

IN 전처리 데이터 + CNN 학습 모델
OUT 학습된 모델

- CNN이 사진의 특징을 추출
- 예측과 정답 비교로 가중치 조정
- 검증 손실이 가장 낮은 모델 보관

epochs 10 · batch 32 · lr 0.001

CNN 모델이란? (합성곱 신경망)

사진을 작은 돋보기로 훑으며 **선 → 모서리 → 모양** 순으로 특징을 쌓아 **"이 사진은 무엇?"**에 답하는 **분류** 모델

오늘 MNIST 학습 노드가 바로 이 CNN이에요

3 배포

cnn_classification (mnist 서빙)

IN 학습된 모델
OUT 서빙 'mnist 서빙 (교육)'

- POST /infer 에 이미지 1장
- 학습과 같은 크기·색상 변환
- /health 로 준비 상태 확인

CPU 4C8GB · GPU 미사용

결과물 **https://aiopen-serving.jntp-ax.or.kr/jntp-cpu/SV-88718065**

숫자 이미지 → 0~9 중 무엇인지

STEP 01

워크스페이스로 들어가기

1. 상단 **[워크스페이스]** 메뉴 클릭
2. **'해커톤 교육 워크스페이스'** 카드 클릭
3. 카드에서 관리자 · 참여자 수 · 크레딧 잔액 · 자산 개수 확인

✓ **확인** 전체 목록 '해커톤 교육 워크스페이스' 카드 1개만 보이면 정상

! 워크스페이스는 팀별 작업 공간
— **크레딧(실행 비용)**과 자원 한도를 함께 가집니다
- 해커톤 팀별 자원 한도 사용
: 8vCPU / 16GiB Mem / GPU 45GiB vRAM



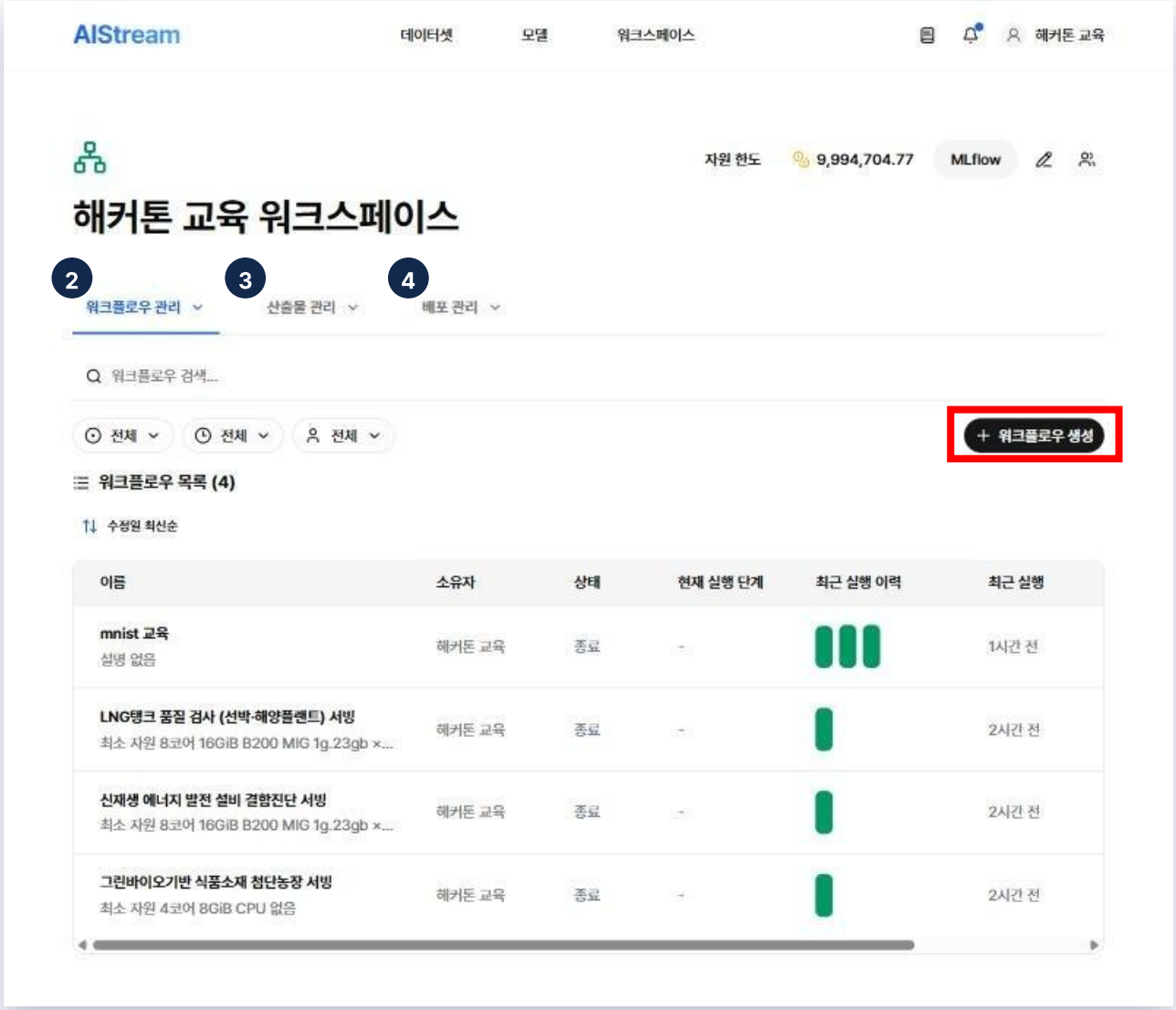
STEP 02

워크스페이스 화면 구성 이해하기

1. 상세 화면은 **워크플로우 관리 · 산출물 관리 · 배포 관리** 3개 탭
2. **워크플로우 관리** — 워크플로우 · 프리셋 · 템플릿
3. **산출물 관리** — 데이터셋 · 등록 모델(버전)
4. **배포 관리** — 배포 목록 · 서빙 URL

✓ **확인** 오른쪽 위 [+ 워크플로우 생성] 위치 확인 → 다음 단계로

✓ **확인** 최근 실행 이력 막대: **초록 = 성공, 빨강 = 실패** (오른쪽이 최신)



시작 전 꼭 확인: 이름 규칙

모든 팀이 **같은 워크스페이스**를 함께 씁니다. 이름으로 **내 워크플로우 · 데이터셋 · 모델 · 서빙**을 구분해요

1	**워크플로우 이름** 워크플로우 생성 › 1단계 기본 정보	mnist_팀번호팀_이름 **mnist_1팀_홍길동**
2	**전처리 데이터셋** 전처리 템플릿 › 출력 › 신규 데이터셋 생성	mnist_전처리_팀번호팀_이름 **mnist_전처리_1팀_홍길동**
3	**학습 모델** 학습 템플릿 › 입력 › 신규 모델 생성	mnist_모델_팀번호팀_이름 **mnist_모델_1팀_홍길동**
4	**서빙** 배포 노드 › 출력 › 신규 서빙 생성	mnist_서빙_팀번호팀_이름 **mnist_서빙_1팀_홍길동**

> 띄어쓰기 대신 **밑줄(_)**로 연결하세요. 이름이 겹치면 다른 사람 결과와 섞여 **내 서빙 URL을 찾기 어려워요**.

STEP 03

워크플로우 새로 만들기

1. 워크플로우 관리 › **[+ 워크플로우 생성]**
2. **워크플로우 이름** 입력 (아래 규칙)
3. 실행 유형 **'단발성'** 선택
4. **[다음으로]**

워크플로우 이름 규칙

mnist_1팀_홍길동

mnist_팀번호팀_이름 · 띄어쓰기 없이 밑줄(_)

✓ **확인** 2단계 **'워크플로우 설정'** 화면으로 이동



STEP 04

프리셋으로 템플릿 노드 한 번에 불러오기

1. 오른쪽 위 **프리셋** › **'mnist 교육 프리셋'** 선택
2. **전처리 → 학습 → 배포** 노드 3개가 자동으로 연결
3. 노드의 **! 표시** = 설정이 필요한 곳

✓ **확인** 상단에 **'설정 확인 0/3'**이 보이면 다음 단계로

! 프리셋에는 **데이터셋 · 모델 · 서빙 연결**이 저장되지 않아 노드마다 직접 선택합니다



STEP 05

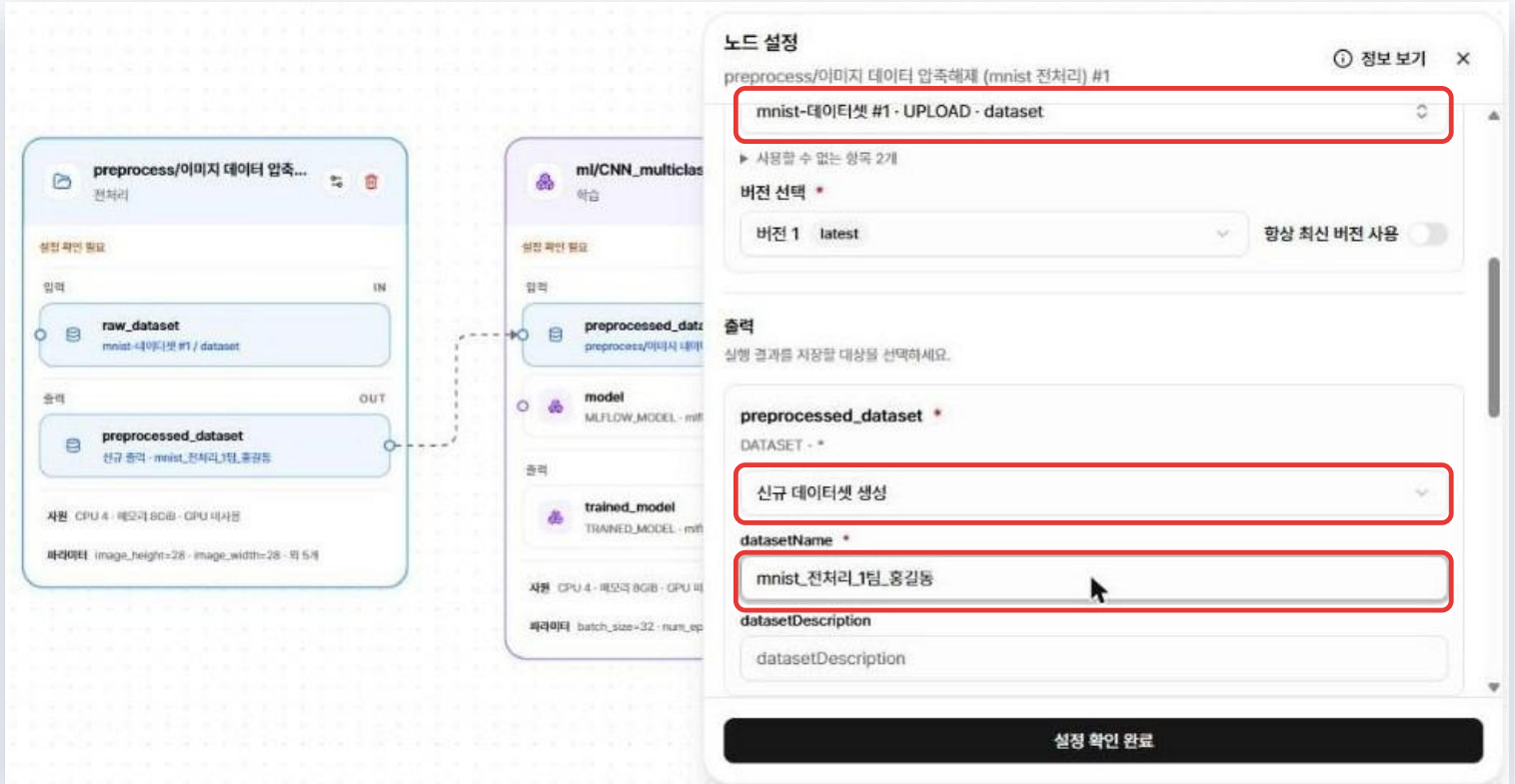
전처리 노드: 데이터 넣고 결과 이름 정하기

1. 전처리 노드 **설정 아이콘** 또는 더블클릭
2. 입력 **raw_dataset** › **'mnist-데이터셋'**
3. 출력 **preprocessed_dataset** › **'신규 데이터셋 생성'**
4. **datasetName** 입력 → **[설정 확인 완료]**

전처리 데이터셋 이름 규칙

mnist_전처리_1팀_홍길동

mnist_전처리_팀번호팀_이름



STEP 06

학습 노드: CNN 모델 선택하기

1. 학습 노드 열기 (ml/CNN_multiclass_classification)
2. 입력 **model** › '신규 모델 생성'
   › modelName 입력 (아래 이름 규칙)
3. 출력 **trained_model**은 자동 설정 (새로 만든 모델에 버전 등록)
4. **[설정 확인 완료]**

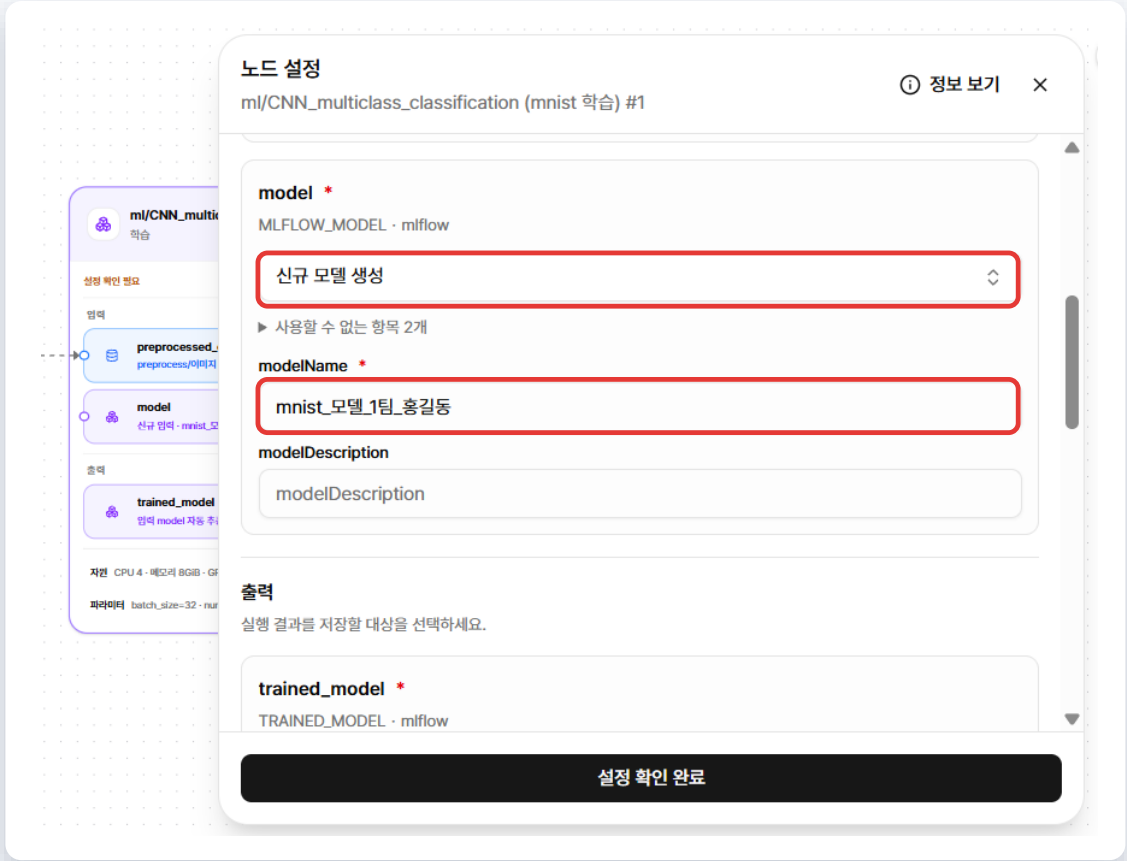
모델 이름 규칙

mnist_모델_1팀_홍길동

mnist_모델_팀번호팀_이름

✓ 확인 상단 **'설정 확인 2/3'**

! preprocessed_dataset 입력은 앞 노드와 **자동 연결** — 따로 고르지 않아요



STEP 07

학습 파라미터와 자원 읽기

1. 학습 노드 설정 창 아래쪽 **실행 파라미터**와 **자원** 확인
2. 오늘은 **바꾸지 않고 기본값** 그대로 실행

num_epochs
10
전체 데이터를 10번 반복 학습

batch_size 32
한 번에 32장씩 묶어서 학습

✓ **확인** 자원 **4C8GB · No GPU**
— 가벼운 CNN이라 CPU로 충분 (분당 4 크레딧)



STEP 08

배포 노드: 서빙 이름 정하고 생성하기

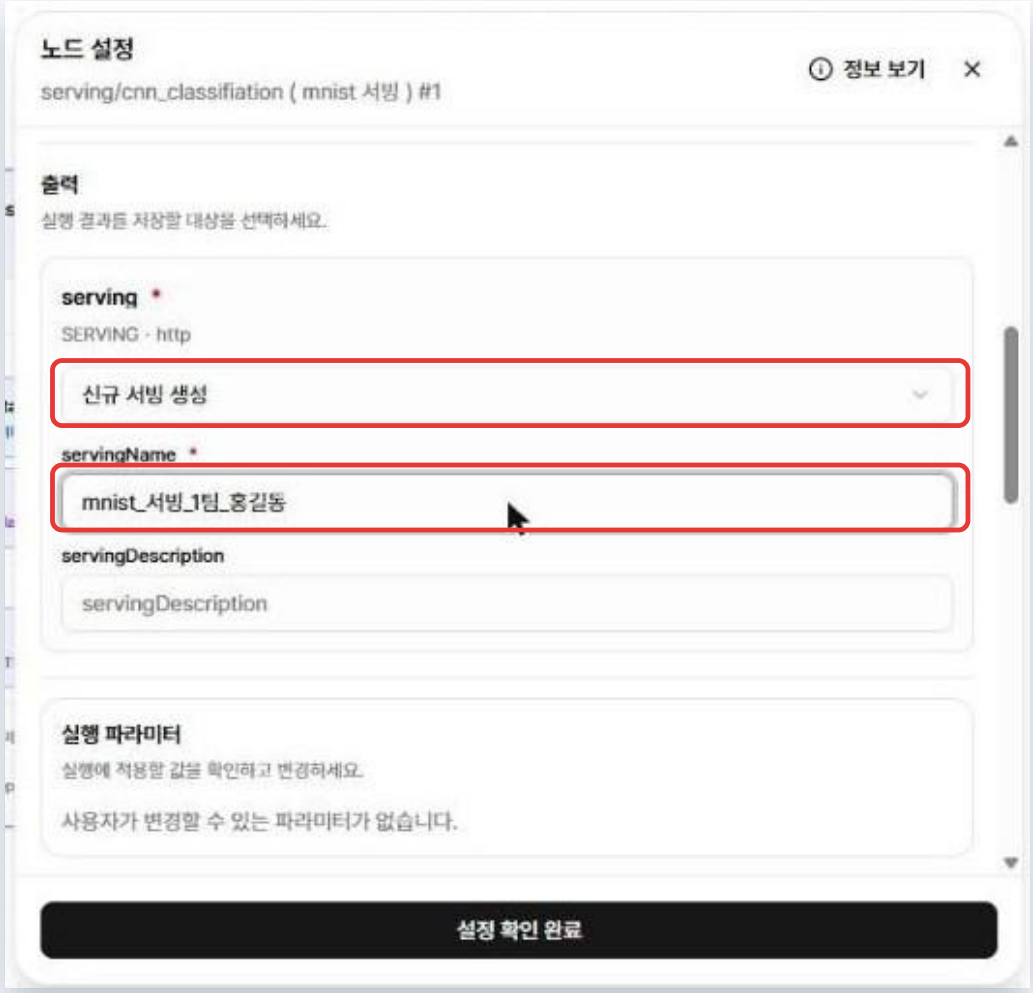
1. 배포 노드 열기 (serving/cnn_classifiation)
2. 출력 **serving** › **'신규 서빙 생성'**
3. **servingName** 입력 → **[설정 확인 완료]**
4. 상단 **'설정 확인 3/3'** → **[생성하기]**

서빙 이름 규칙

mnist_서빙_1팀_홍길동

mnist_서빙_팀번호팀_이름 · 배포 목록에서 이 이름으로 찾아요

✓ **확인** [생성하기]를 누르면 **내 워크플로우**가 목록에 생깁니다



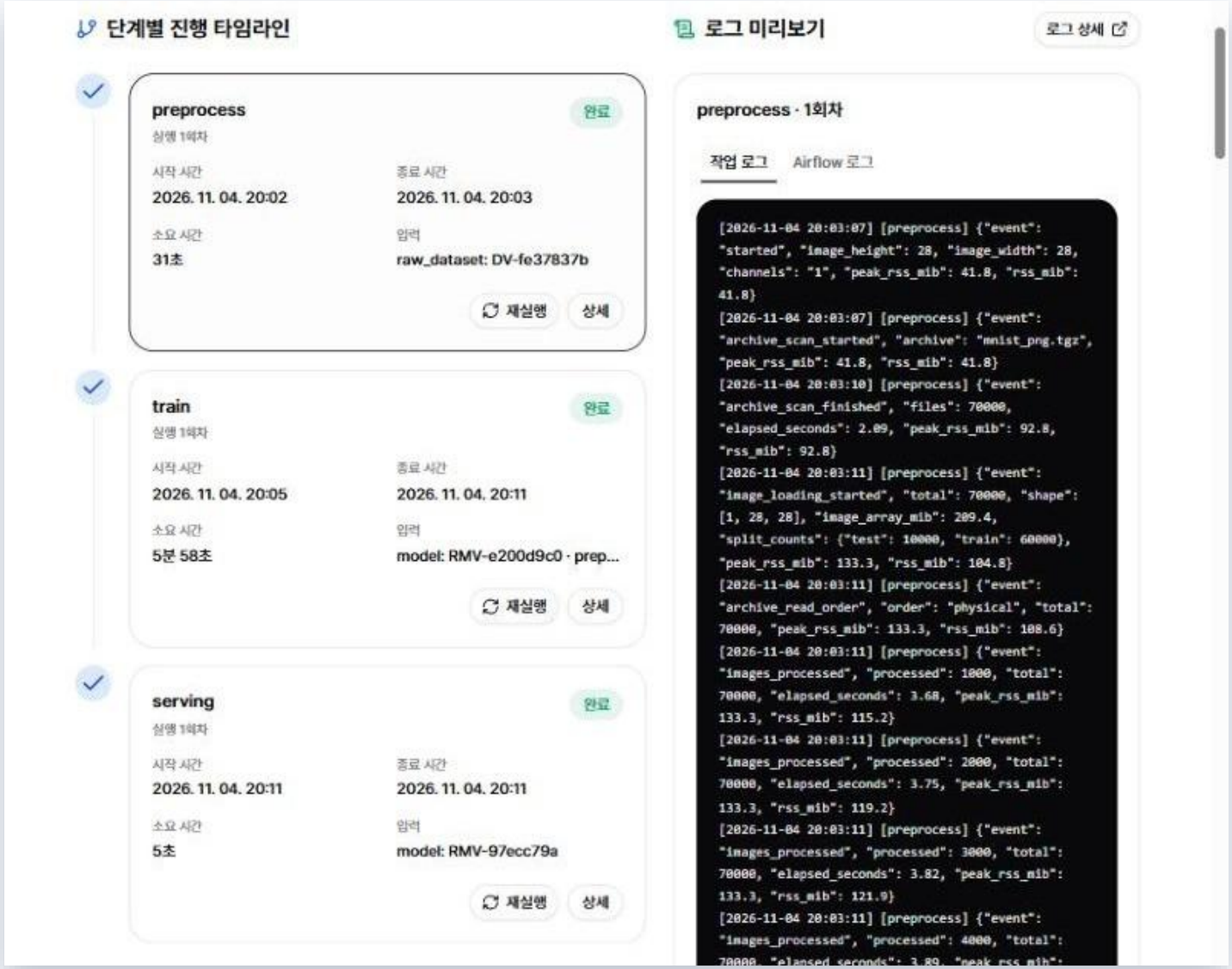
STEP 09

▶ 실행하고 진행 확인하기

1. 워크플로우 상단 ▶ **[워크플로우 실행]** 클릭
2. 실행 이력에 생긴 새 회차 클릭
3. 단계별 진행 확인

preprocess	약 30초 · 숫자 이미지를 학습용으로 정리
train	약 6분 · 숫자 이미지 학습 검증 진행
serving	약 5초 · 끝나면 배포 목록에 생성

✓ **확인** 세 단계가 모두 **'완료'**면 성공



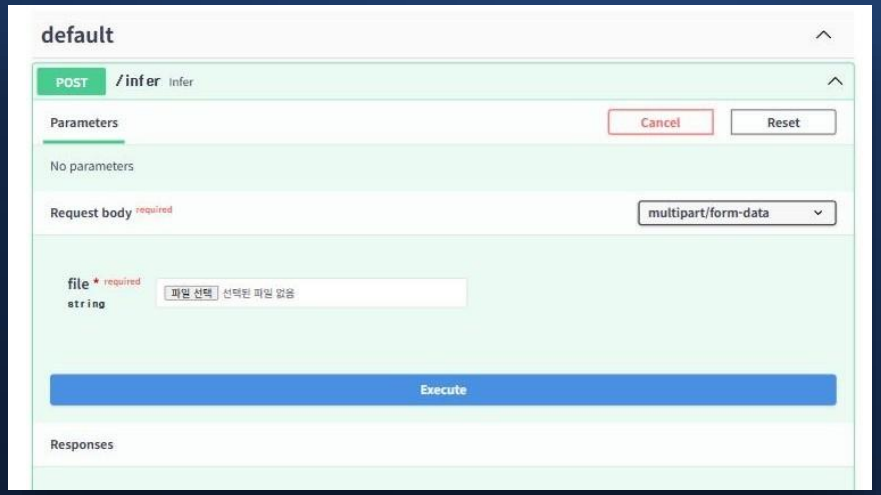
AIStream 2 · 04 실습 2

서빙 API로 서비스에 활용하기

학습 대기 중: URL 구조 · 연결 구조 → 학습 완료 후: 내 서빙 URL 확인 → /docs로 API 읽기 → Codex로 내 서비스 만들기 → 직접 그려서 숫자 추론

코드 없이 화면에서

Codex로 서비스 만들기



내 서비스에 연결하는 구조



직접 호출은 막혀 있음

웹 화면에서 바로 부르면 브라우저가 차단해요

주소를 숨김

서빙 URL을 화면에 노출하지 않음

후처리를 한곳에서

결과 저장 · 알림 등을 백엔드에서

서빙 URL 구조 한눈에 보기

공통 주소 + 서빙 ID + 기능 경로 — 해커톤 중에는 **서빙 ID만 바꿔 가며** 사용합니다.

https://aiopen-serving.jntp-ax.or.kr/jntp-cpu/SV-88718065/infer

서빙 서버 주소 (공통) | 서빙 ID (배포마다 다름) | 기능 경로

GET	/health	살아 있는지 확인	GET	/docs	Swagger 대화형 문서
GET	/openapi.json	개발용 API 명세	POST	/infer	실제 AI 추론 요청

MNIST 서빙에서 확인한 구성입니다. 다른 서빙은 **/docs**에서 실제 경로를 확인하세요.

STEP 01

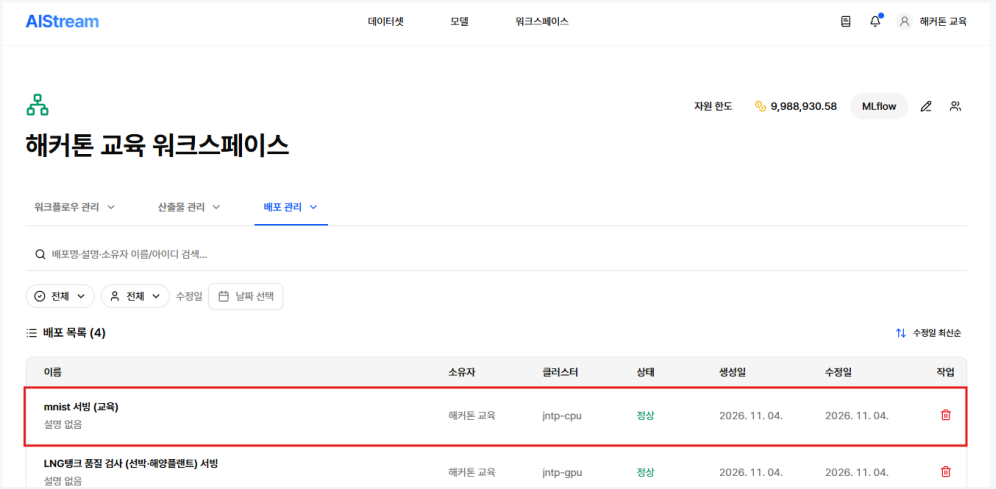
학습이 끝나면 내 서빙 URL 확인

1. 워크스페이스 **[배포 관리]** 탭
2. **'mnist 서빙 (교육)'** 클릭
3. 운영 상태 **'정상'**, 승인 상태 **'승인 완료'** 확인
4. **서빙 URL 복사 (화면 표기: 서비스 URL) → 다음 단계에서 사용**

예시 — 오늘 실습 서빙 주소

```
> https://aiopen-serving.jntp-ax.or.kr/jntp-cpu/SV-88718065
```

복사





STEP 02

/docs에서 내 API 설명서 읽기

1. 내 서빙 URL 뒤에 **/docs**를 붙여 새 탭에서 열기 (배포 상세 [Swagger 열기]와 같은 화면)
2. 이 서비스로 **무엇을 할 수 있는지** 적힌 설명서 — 오늘은 **읽기만** 해요

GET **/health** "살아 있나요?"
모델이 준비됐는지 확인 · 정상이면 **{"status":"ok"}**

POST **/infer** "이 사진 뭐예요?"
이미지를 보내면 **예측 결과**를 돌려주는 핵심 기능

GET = 정보를 **가져오기** · POST = 데이터를 **보내기**



STEP 03

/infer: 무엇을 보내고 무엇을 받을까?

① 보내는 것 · Request body

file (필수) — 숫자 이미지 **1장**

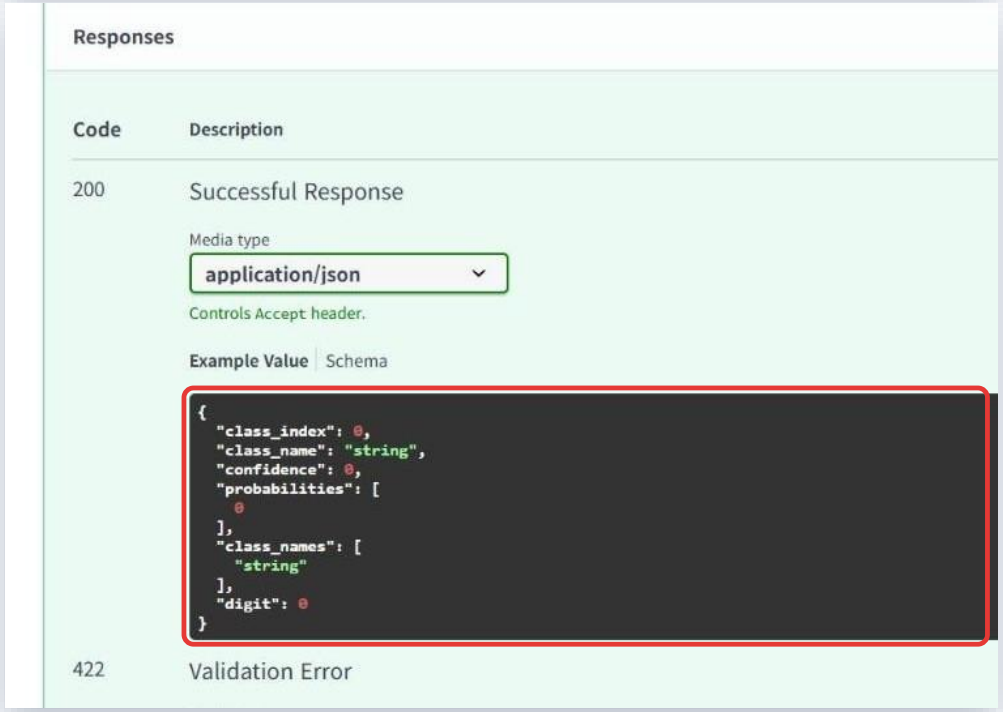
multipart/form-data = 파일을 첨부해서 보내는 방식

② 받는 것 · Responses 200

`class_name`	예측한 숫자 (예: "7")
`confidence`	확신 정도 0~1 (0.99 ≈ 99%)
`probabilities`	0~9 숫자별 확률 10개
`class_names`	모델이 아는 정답 목록 (0~9)

422 = 보낸 형식이 틀렸을 때 (예: file 없이 요청)





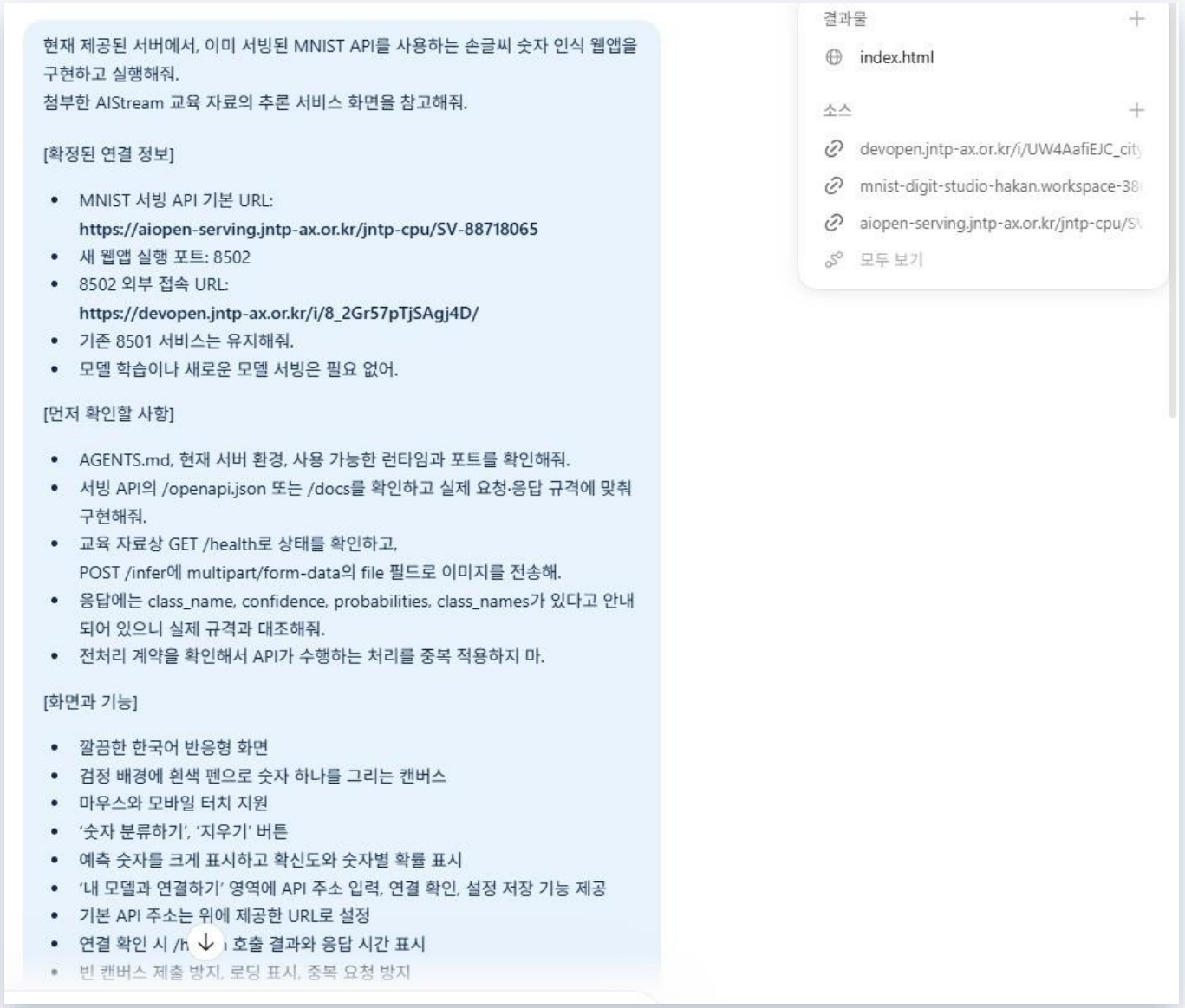
STEP 04

Codex로 내 웹 서비스 만들기

1. 팀 개발 서버에 연결된 **Codex**에서 새 작업 열기
2. 아래 **요청서 틀 5칸**에 맞춰 프롬프트 작성
3. Codex가 **확인 → 구현 → 테스트 → 실행**까지 진행

요청서(프롬프트) 틀

1	**연결 정보**	서빙 URL · 실행 포트 · 서비스 URL(외부 접속)
2	**사전 확인**	/docs · openapi.json 규격, /health · /infer 형식
3	**화면 · 기능**	캔버스, 분류 · 지우기 버튼, 결과 · 오류 안내
4	**구현 · 실행**	폴더 구조, 테스트, 0.0.0.0:포트로 실행
5	**완료 기준**	실제 API 예측 성공, 외부 URL 접속 확인

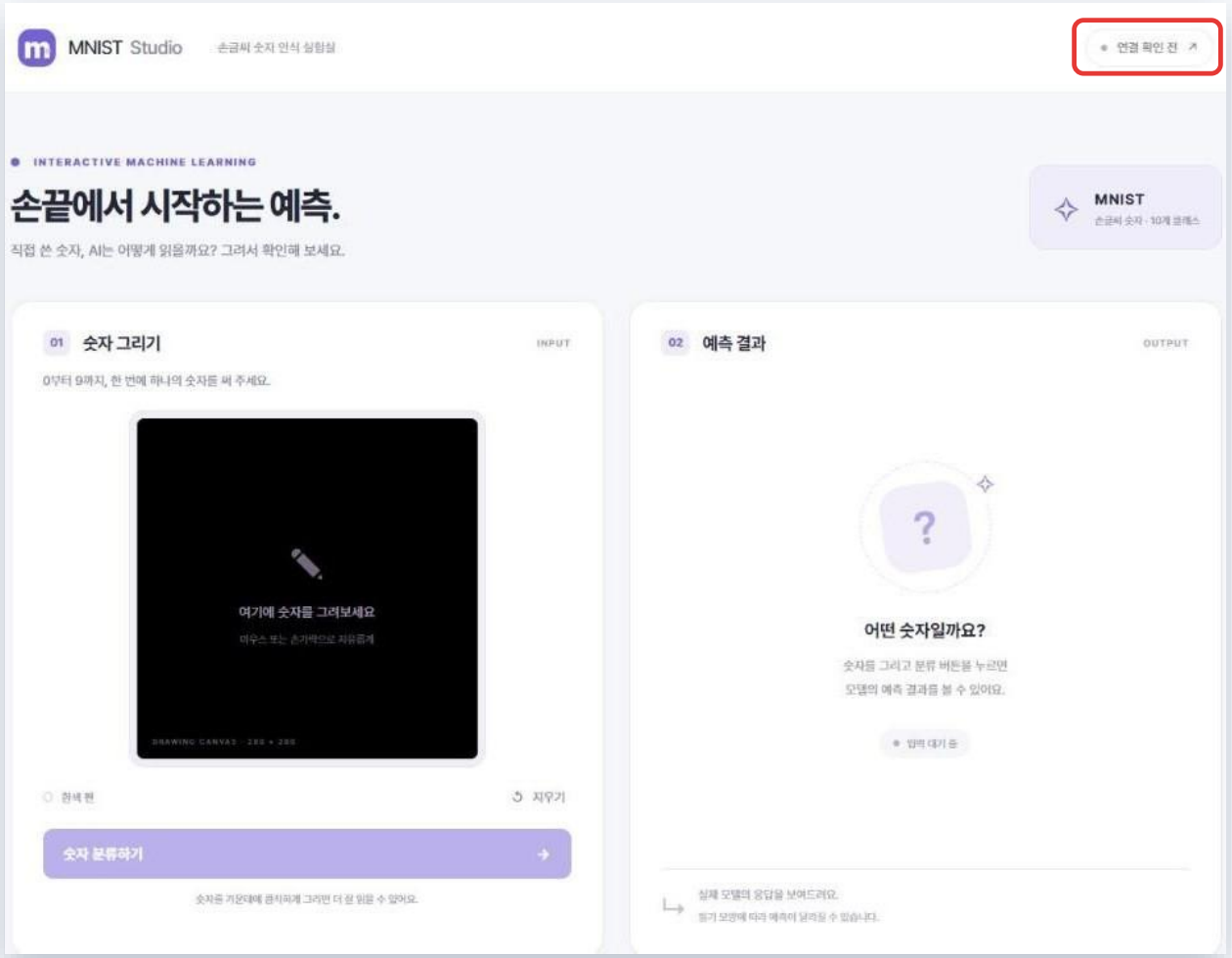


STEP 05

서비스 URL로 내 서비스 열기

1. Codex가 알려준 서비스 URL(외부 접속 주소) 열기 (예: https://devopen.jntp-ax.or.kr/i/.../)
2. 화면 구성 확인: **① 숫자 그리기 · ② 예측 결과**
3. 오른쪽 위 **연결 상태** 표시 확인
4. 아래쪽 **'내 모델과 연결하기'**에서 서빙 연결 (다음 단계)

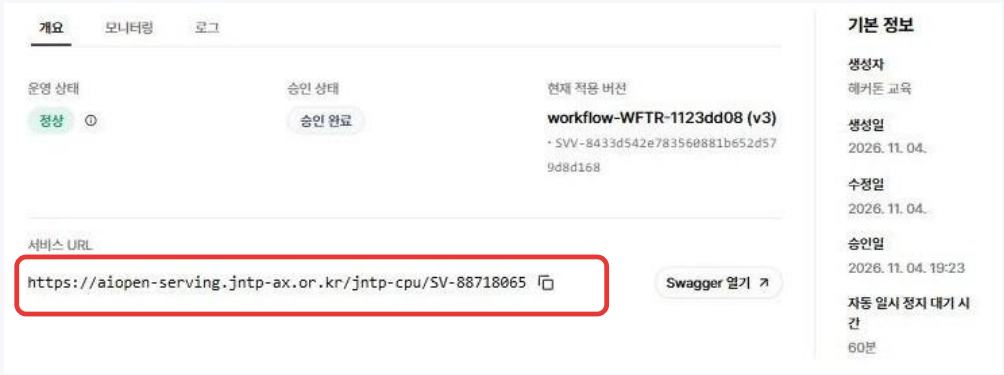
> 서비스를 직접 만들기 어렵다면, 아래 링크로 들어오세요.
> https://devopen.jntp-ax.or.kr/i/8_2Gr57pTjSAgj4D/



STEP 06

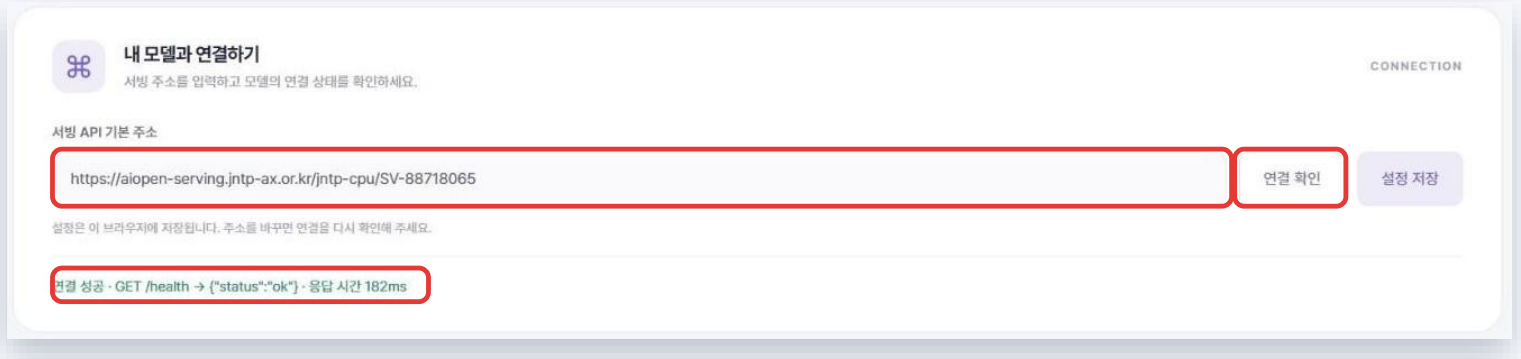
내 서비스에 서빙 연결 확인하기

1. 화면 아래 **'내 모델과 연결하기'**
2. **서빙 API 기본 주소**에 내 서빙 URL 확인 (기본값 입력됨)
3. **[연결 확인]** → **[설정 저장]**



✓ 확인 **연결 성공 · GET /health → {"status":"ok"}** · 응답 시간 182ms

! 주소를 바꾸면 **연결 확인**을 다시 눌러 주세요 · 설정은 이 브라우저에 저장돼요



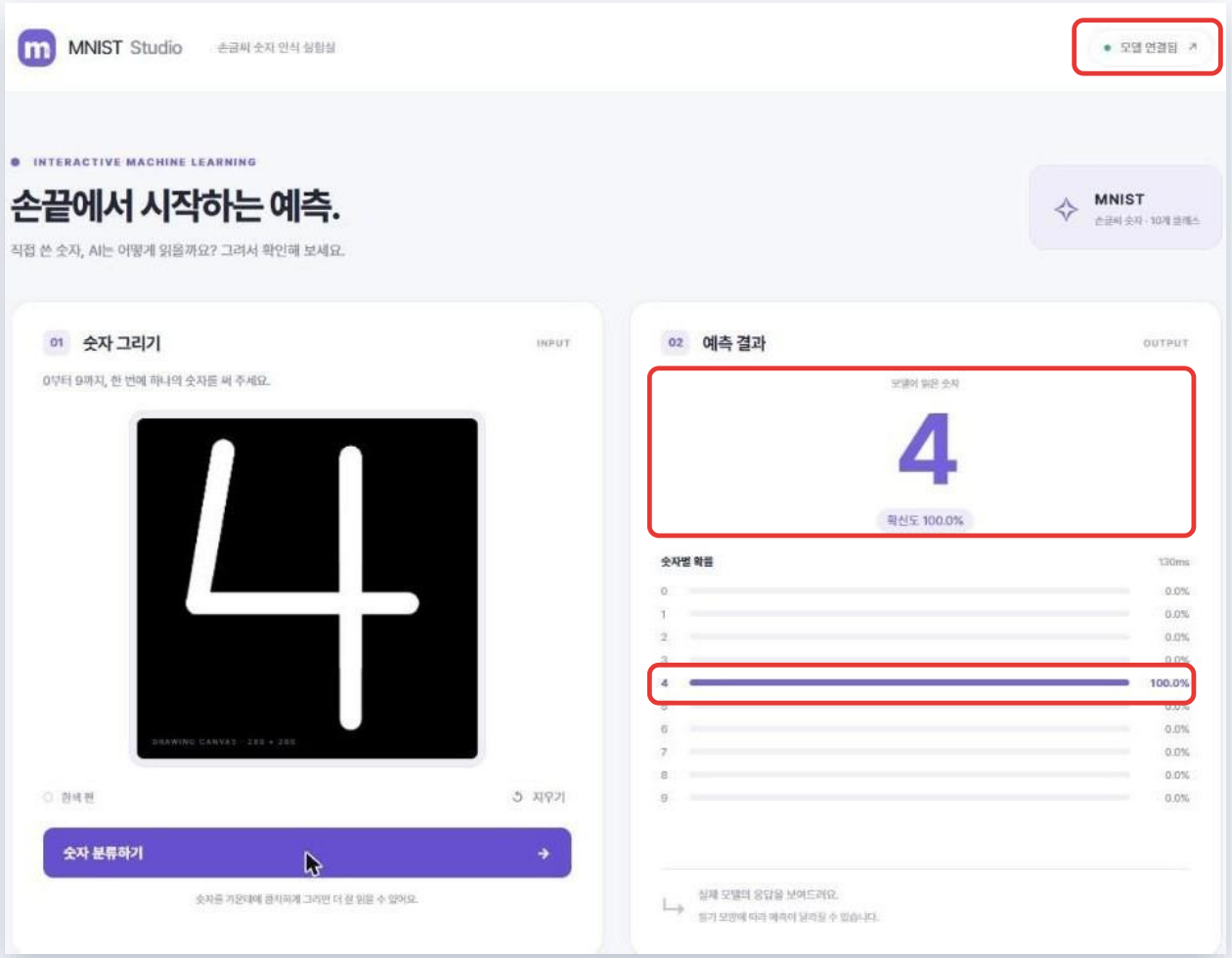
STEP 07

숫자를 직접 그려서 추론해 보기

1. **숫자 그리기** 캔버스에 숫자 하나를 가운데 크게
2. **[숫자 분류하기]** 클릭
3. 오른쪽 **예측 숫자 · 확신도 · 숫자별 확률** 확인
4. 다시 하려면 **[지우기]**

✓ **확인** 실제 결과: **4**를 그려 예측 **4 · 확신도 100%** (응답 130ms)

! 숫자 **하나만** 그려요 · 여러 자리 숫자는 판별하지 않아요



05 해커톤 사전 안내

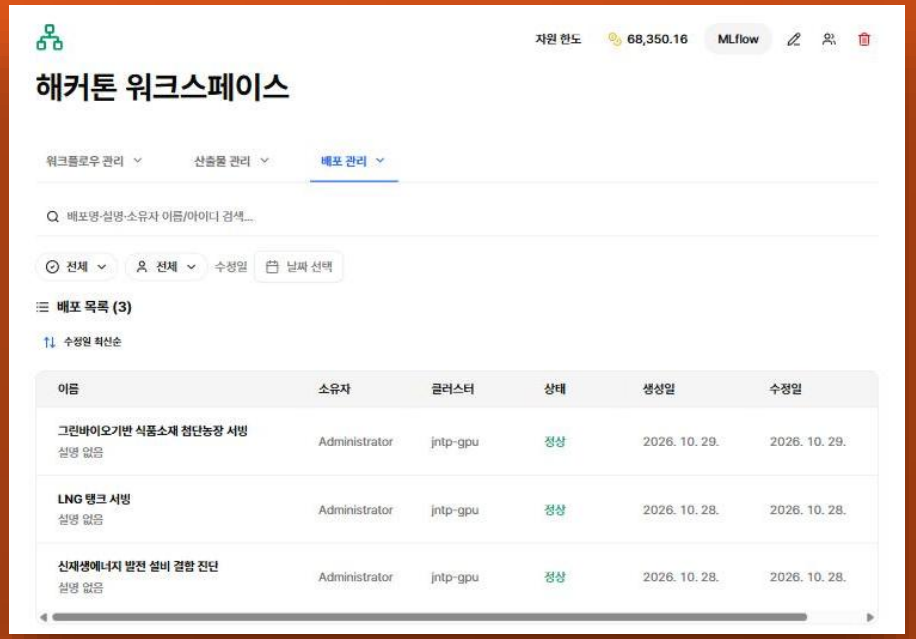
해커톤 문제와 제공 서빙 미리 보기

어떤 트랙을 고르고, 어떤 서빙 API를 쓰는지 미리 확인해요

A 트랙 서빙 3종

B 트랙 자사 모델

11. 7.(토) ~ 11. 8.(일)



트랙 고르기: A 제공 모델 활용형 · B 자사 모델/데이터형

소재·부품·장비 / 바이오 / 에너지 중 **1개 분야**를 고르고, 해결할 문제는 **팀이 직접 정의**합니다

A 트랙 제공 모델 활용형

- 해커톤에서 제공하는 **데이터 · 학습 모델** 활용
- AIStream에 배포된 **서빙 API 3종** 중 선택해 호출
- 오늘 실습 2와 **같은 방식** (서빙 URL · /docs · 웹 서비스)

활용 데이터: 제공 샘플 데이터셋 (AI Hub 원천 데이터 기반)

B 트랙 자사 모델/데이터형

- 참가 기업이 **보유한 데이터 · 모델**로 자사 현안 해결
- 자사 모델을 AIStream에 **등록 · 서빙** (부록 ① 참고)
- 해커톤 제공 모델 · 도구 **함께 사용 가능**

팀 자원 한도 안에서 구동되는지 **사전에 확인**해 주세요

외부 데이터(공공데이터 등)를 **새로 수집 · 활용할 수 없어요** — 자사 데이터와 해커톤 제공 데이터만 사용합니다.

A 트랙 제공 서빙 3종

세 모델 모두 **AIStream에 배포된 서빙 API**로 제공됩니다 · 해커톤 워크스페이스 › **배포 관리**에서 확인

소재 · 부품 · 장비

LNG 탱크 품질 검사

제공 데이터 · 모델

LNG 탱크 품질 검사 영상 데이터로 학습한 모델

배포 목록 이름

LNG탱크 품질 검사 (선박·해양플랜트) 서빙

이런 서비스를 만들 수 있어요

검사 결과 수집 → 이상 징후 대시보드

참고 주제
품질검사 자동 판정 · 불량 관리

바이오

그린바이오 기반 식품소재 첨단농장

제공 데이터 · 모델

그린바이오 기반 식품소재 첨단농장 데이터로 학습한 모델

배포 목록 이름

그린바이오기반 식품소재 첨단농장 서빙

이런 서비스를 만들 수 있어요

작물 사진 → 생육 현황 · 수확 시기 알림

참고 주제
생육단계 판별 · 수확 의사결정

에너지

신재생 에너지 발전 설비 결함진단

제공 데이터 · 모델

풍력 · 태양광 이미지 결함 분석 모델 (wind / solar)

배포 목록 이름

신재생 에너지 발전 설비 결함진단 서빙

이런 서비스를 만들 수 있어요

설비 사진 → 결함 진단 · 점검 리포트

참고 주제
신재생 설비 정비 의사결정

꼭 확인 **서빙마다 기능 경로와 입력 형식(이미지 · 수치 등)이 달라요 → 각 서빙의 /docs를 먼저 읽기**

해커톤 **당일**

배포 목록에서 서빙 찾기

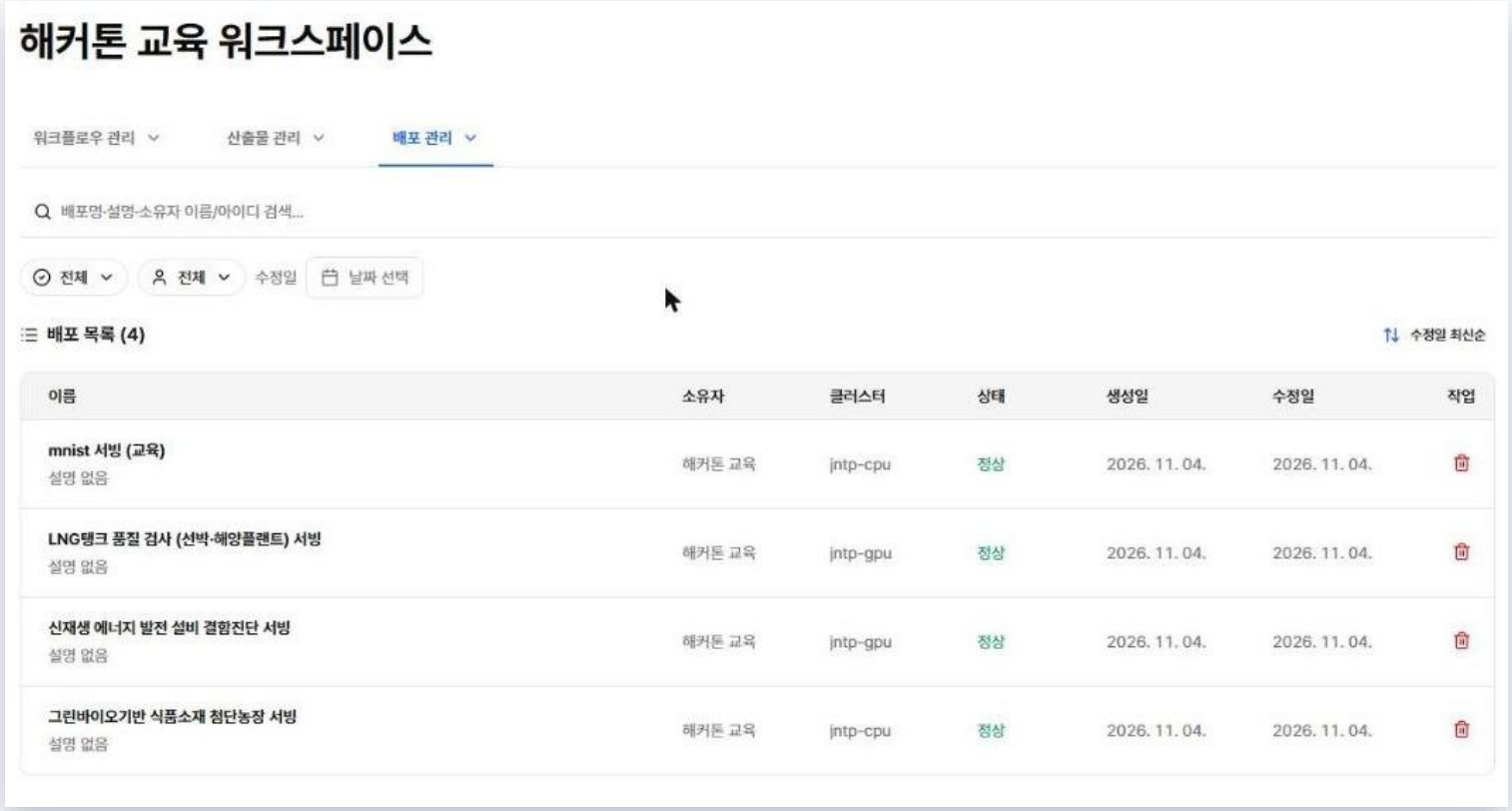
1. 해커톤 당일 안내받는 워크스페이스 › **[배포 관리]** 탭
2. 팀이 쓸 서빙 클릭 → 운영 상태 확인
3. 서빙 URL 복사 → /docs에서 POST 경로 · 입력 형식 확인

그린바이오기반 식품소재 첨단농장 서빙
.../jntp-cpu/SV-c562094f · POST /predict

LNG탱크 품질 검사 (선박·해양플랜트) 서빙
.../jntp-gpu/SV-d4074093 · POST /predict

신재생 에너지 발전 설비 결함진단 서빙
.../jntp-gpu/SV-e434a471 · POST /predict/wind · /solar

! 서빙마다 POST 경로와 입력 형식이 달라요



추가 기능 ① 내 모델 등록하기

워크스페이스 › **산출물 관리 › 모델** · 자사 모델(B 트랙)이나 직접 학습한 가중치를 AIStream에 올려 배포에 씁니다

1 [모델 생성하기]

모델명 · 설명을 입력해 빈 등록 모델을 만듭니다



2 [새 버전 생성]

모델 상세에서 **가중치 · 모델 산출물 파일**을 업로드 (여러 파일 가능)



3 워크플로우에서 사용

배포 · 학습 노드의 **model 입력**에서 등록한 모델과 버전을 선택해 서빙합니다



또는 **[모델 가져오기] — 접근 가능한 모델 버전을 복제해 내 워크스페이스의 새 모델로 가져오기**

추가 기능 ② 템플릿 생성하기

워크스페이스 › **워크플로우 관리 › 템플릿** · 내 코드를 노드로 만들어 워크플로우에서 다시 씁니다

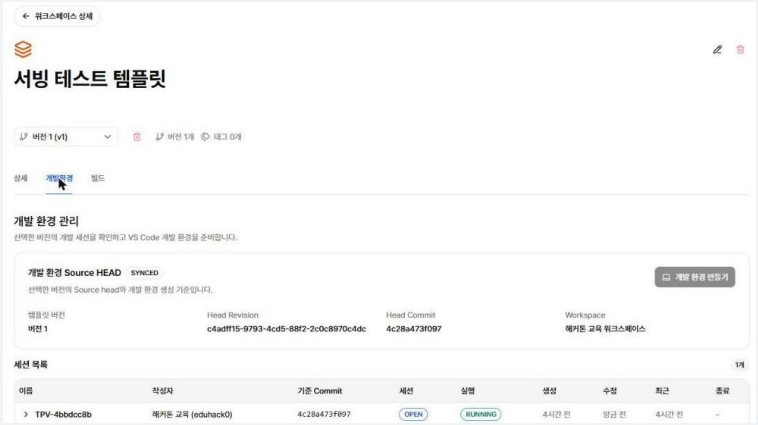
1 [템플릿 생성]

이름 · 유형(전처리 / 모델 / 배포) · 설명을 입력해 빈 템플릿 생성



2 개발환경

템플릿 상세 › 개발환경 › **[개발 환경 만들기]** → VS Code에서 코드 작성



3 빌드 → 게시

빌드가 끝나면 게시 → 워크플로우 오른쪽 카탈로그에서 **노드로 추가**



참고 워크플로우에는 스텝(빌드 완료된 코드)이 있는 템플릿만 사용할 수 있습니다

Q&A

감사합니다

막히면 언제든 운영진에게 질문해 주세요 · AIStream aiopen.jntp-ax.or.kr · 사용자 가이드: AIStream 상단 문서 아이콘